**Universidad de San Carlos de Guatemala**

**Centro Universitario de Occidente (CUNOC)**

División de Ciencias de la Ingeniería

Ingeniería en Ciencias y Sistemas

Sistemas Operativos 2

# MANUAL TÉCNICO

## Centro de Distribución Concurrente REDXela

**Proyecto Final de Sistemas Operativos 2**

Modelado de Planificación de CPU, Exclusión Mutua con Semáforos, Comunicación entre Procesos (IPC) con Buffer Acotado, Asignación de Almacenamiento No Contiguo (Paginación) y Detección/Resolución de Interbloqueos (Deadlock).

**Estudiante:** Elmer Miguel

**Registro Académico:** 201931295

**Quetzaltenango, Guatemala, Octubre de 2026**

# Índice


**1. Introducción y Alcance del Sistema** **2**

**2. Arquitectura del Sistema y Patrones de Software** **2**
2.1. Diagrama de Arquitectura por Capas . . . 3
2.2. Patrones de Diseño Aplicados . . . 3

**3. Modelo de Dominio y Diagrama de Clases** **3**

**4. Gestión de Recursos Compartidos y Sincronización** **4**
4.1. Especificación de Recursos y Capacidades . . . 4
4.2. Justificación Teórica del Semáforo con Equidad . . . 5
4.3. Adquisición Atómica y Prevención de Retención Indefinida . . . 5

**5. Planificación de Pedidos y Políticas de Atención** **5**
5.1. Ciclo de Vida y Transición de Estados del Pedido . . . 5
5.2. Políticas de Planificación Implementadas . . . 5
5.3. Niveles de Servicio y SLAs Contractuales . . . 6

**6. Comunicación entre Procesos (IPC) con Buffer Acotado** **6**
6.1. Modelo Productor-Consumidor sobre Buffer Acotado . . . 6
6.2. Mecanismos de Sincronización y Contrapresión . . . 7

**7. Almacenamiento Físico y Asignación No Contigua (Paginación)** **7**
7.1. Estructura Físico-Espacial del Almacén . . . 7
7.2. Algoritmo de Fragmentación y Tabla de Asignación . . . 7

**8. Conflicto de Recursos: Detección y Resolución de Deadlock** **8**
8.1. Las Cuatro Condiciones de Coffman . . . 8
8.2. Grafo de Asignación de Recursos y Detección de Ciclos . . . 8
8.3. Protocolo de Resolución Manual . . . 9

**9. Interfaz Gráfica de Usuario y Soporte LookAndFeel** **9**
9.1. Pestaña 1: Monitoreo General (Recursos y Pedidos) . . . 9
9.2. Pestaña 2: Almacén Físico e Inventario . . . 10
9.3. Pestaña 3: Estadísticas y Evaluación de Políticas . . . 11
9.4. Diálogos Modales del Sistema . . . 11

**10. Estrategia de Pruebas y Verificación Automatizada** **12**
10.1. Matriz de Pruebas Unitarias y de Integración . . . 12
10.2. Resultado de la Ejecución de Pruebas . . . 12

**11. Conclusiones Técnicas y Recomendaciones** **13**

# 1. Introducción y Alcance del Sistema

El presente documento constituye el manual técnico formal del sistema de simulación del centro de distribución de comercio electrónico REDXela. En los sistemas de distribución modernos, la recepción de suministros, la preparación de bultos y el despacho no operan como cadenas lineales simples, sino como entornos altamente concurrentes donde múltiples procesos compiten de forma continua por un conjunto estrictamente limitado de recursos físicos y humanos.

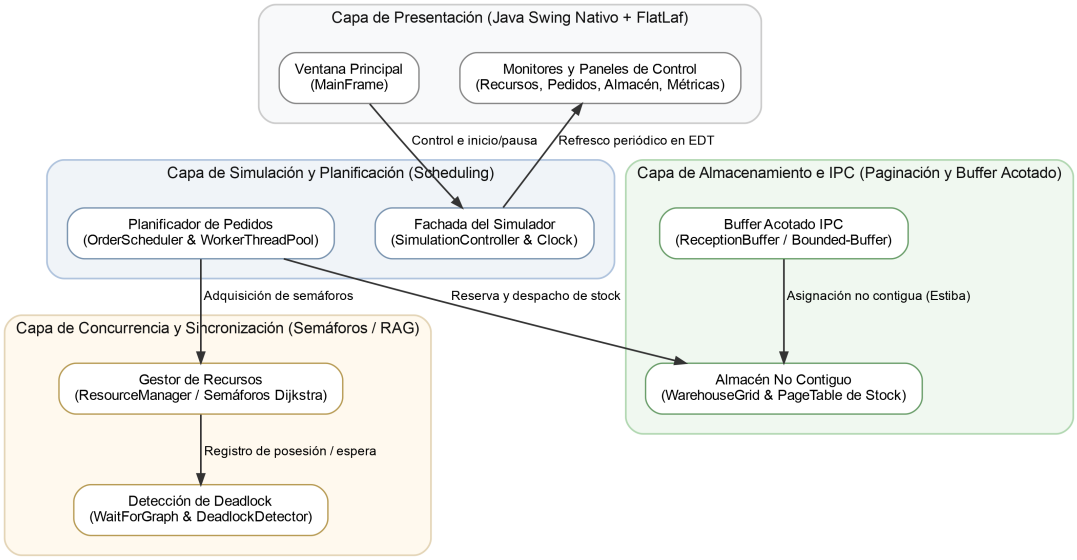
El objetivo central del proyecto es implementar, evaluar y demostrar los principios fundamentales de la teoría de Sistemas Operativos 2 en un entorno práctico desarrollado sobre Java 21 LTS y Java Swing nativo:

- Planificación de Procesos (CPU Scheduling): Coordinación y despacho de pedidos evaluando múltiples políticas de atención (Prioridad Estricta, Aging, Shortest Job First y FIFO) y comprobando el impacto en los acuerdos de nivel de servicio (SLA).
- Sincronización y Exclusión Mutua: Administración atómica de seis recursos compartidos finitos mediante semáforos contadores de Dijkstra con política de equidad (fairness).
- Comunicación entre Procesos (IPC): Canal asíncrono y desacoplado entre el área de Recepción de Mercadería y el área de Control de Inventario utilizando el patrón Productor-Consumidor sobre un buffer acotado, evitando la interferencia directa de estructuras internas.
- Administración de Almacenamiento Físico: Implementación de almacenamiento no contiguo análogo a la paginación de memoria virtual, permitiendo fragmentar lotes de producto en casillas tridimensionales dispersas según la disponibilidad.
- Detección y Resolución de Interbloqueo (Deadlock): Representación formal mediante Grafos de Asignación de Recursos (RAG / Wait-For Graph), detección de ciclos dirigidos por búsqueda en profundidad (DFS) y resolución manual mediante apropiación controlada y reversión a cola de espera.
- Arquitectura Gráfica Desacoplada: Interfaz en Java Swing nativo sin esquemas de color fijos, gobernada enteramente por el motor de estilos LookAndFeel (FlatLaf).

# 2. Arquitectura del Sistema y Patrones de Software

El diseño de la aplicación se fundamenta en una separación estricta de responsabilidades organizada en cuatro capas funcionales, asegurando que la lógica de concurrencia sea completamente independiente de la interfaz gráfica de usuario.

### 2.1. Diagrama de Arquitectura por Capas




Arquitectura general por capas del simulador REDXela.

### 2.2. Patrones de Diseño Aplicados

- Patrón Fachada (Façade): Encarnado en la clase SimulationController. Unifica el ciclo de vida del reloj virtual, la parada y arranque de hilos trabajadores, la inserción de eventos manuales y la recolección de métricas, proveyendo a la interfaz gráfica un punto de acceso único y simplificado.
- Patrón Productor-Consumidor: Implementado a través de ReceptionBuffer. El hilo ReceptionWorker actúa como productor de lotes provenientes de camiones externos, mientras que InventoryWorker consume y reclasifica la mercadería en el almacén de forma asíncrona.
- Patrón Observador (Observer / Listener): Utilizado en ResourceListener y en la notificación de incidentes de interbloqueo. Permite propagar cambios de asignación física hacia la interfaz de usuario de manera desacoplada mediante invocaciones seguras en el Event Dispatch Thread (EDT).
- Patrón Estrategia (Strategy): Modelado en la interfaz OrderComparator y el enumerado SchedulingPolicy. Permite alternar en caliente el algoritmo de ordenamiento de la cola de pedidos sin reiniciar la simulación ni perder datos.

## 3. Modelo de Dominio y Diagrama de Clases

La estructura de clases refleja con fidelidad los términos operativos y las abstracciones teóricas del sistema. A continuación se presenta el diagrama de clases de los módulos centrales.

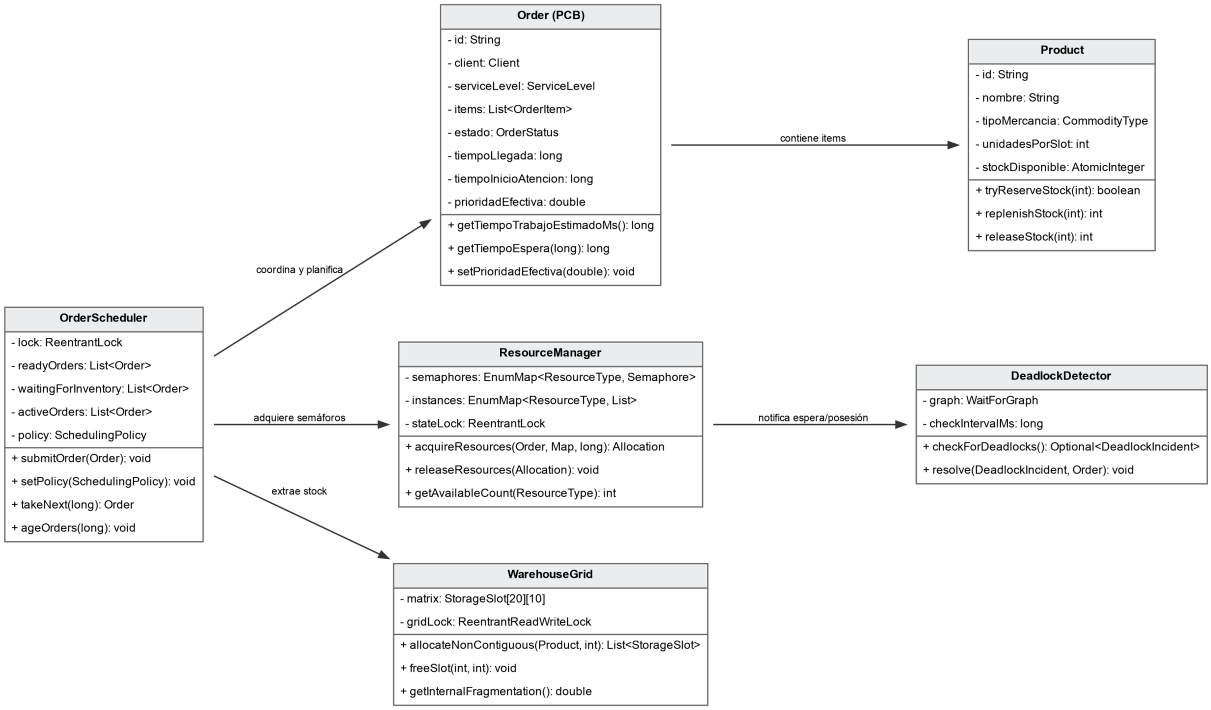



Diagrama de clases principal para la coordinación de pedidos y recursos.

# 4. Gestión de Recursos Compartidos y Sincronización

El centro de distribución cuenta con un conjunto finito de equipos y personal operativo. En términos de sistemas operativos, cada equipo corresponde a una instancia de un tipo de recurso finito sujeto a contención.

## 4.1. Especificación de Recursos y Capacidades

| Tipo de Recurso | Capacidad | Mecanismo de Exclusión y Justificación |
|---|---|---|
| Estaciones de Empaque | 6 unidades | Semáforo contador con equidad (fairness = true). Controla los puestos donde se preparan y embalan físicamente los pedidos. |
| Áreas de Carga | 2 unidades | Semáforo contador de 2 permisos. Bahías de estacionamiento de camiones de despacho y transferencias masivas. |
| Encargados de Bodega | 10 unidades | Semáforo contador de 10 permisos. Personal de recolección y manipulación de artículos en los pasillos del almacén. |
| Montacargas | 3 unidades | Semáforo contador de 3 permisos. Maquinaria pesada requerida exclusivamente para carga pesada y pallets bulk. |
| Estaciones de Calidad | 4 unidades | Semáforo contador de 4 permisos. Puestos de inspección obligatoria para artículos frágiles y de alta tecnología. |
| Sistema de Escaneo de Códigos | 1 unidad | Semáforo binario / Mutex estricto. Dispositivo crítico compartido en exclusión mutua entre el área de recepción y el despacho. |

### 4.2. Justificación Teórica del Semáforo con Equidad

En Java, el constructor de la clase Semaphore acepta un parámetro booleano de equidad:

Semaphore(int permits, boolean fair)

Cuando se inicializa con el valor verdadero, el semáforo garantiza una política FIFO estricta entre los hilos bloqueados que esperan adquirir permisos. En un sistema sin equidad, un hilo recién llegado podría superar a hilos que llevan largo tiempo esperando si el recurso se libera exactamente cuando el nuevo hilo ejecuta tryAcquire. El uso de semáforos con equidad elimina formalmente este comportamiento arbitrario y previene la inanición de hilos en escenarios de alta carga concurrente.

### 4.3. Adquisición Atómica y Prevención de Retención Indefinida

Para evitar situaciones de bloqueo permanente causadas por la adquisición parcial de recursos, el método acquireResources de ResourceManager implementa un protocolo seguro:

1. Valida y normaliza el mapa de requerimientos del pedido según su tipo de mercancía.
2. Define un plazo de vencimiento (deadline) calculado en nanosegundos relativos.
3. Intenta adquirir secuencialmente las unidades de cada tipo de recurso necesario mediante llamadas a tryAcquire con el tiempo restante disponible.
4. Si cualquiera de las solicitudes falla por timeout o el hilo es interrumpido, se ejecuta inmediatamente una rutina de rollback que devuelve en orden inverso todos los permisos adquiridos parcialmente hasta ese momento.

Este mecanismo neutraliza la segunda condición de Coffman (retención y espera indefinida), garantizando que ningún hilo mantenga recursos bloqueados si no es capaz de obtener la totalidad del conjunto requerido para procesar el pedido.

## 5. Planificación de Pedidos y Políticas de Atención

Cada pedido admitido en el sistema actúa como un Bloque de Control de Proceso (PCB). El planificador debe resolver la contención entre múltiples pedidos simultáneos decidiendo qué orden debe ser atendida en cada instante.

### 5.1. Ciclo de Vida y Transición de Estados del Pedido

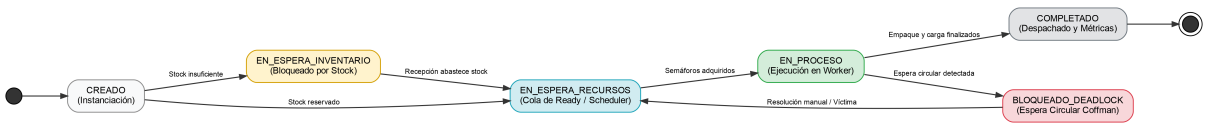



Diagrama de transición de estados del pedido en el simulador.

### 5.2. Políticas de Planificación Implementadas

El sistema implementa cuatro algoritmos clásicos de planificación conmutables dinámicamente mediante el selector de políticas de la interfaz gráfica:

1. Prioridad Estricta por Nivel de Servicio: Los pedidos se atienden rigurosamente según el nivel de servicio contratado (Nivel 1 Exprés $>$ Nivel 2 Prioritario $>$ Nivel 3 Estándar $>$ Nivel 4 Programado $>$ Nivel 5 Económico). A igual nivel, se desempata por orden de llegada (FIFO).

2. Prioridad con Envejecimiento (Aging): Para resolver la inanición de pedidos económicos durante períodos de alta saturación de pedidos exprés, la prioridad efectiva se recalcula periódicamente según la siguiente expresión:

$$P_{\text{efectiva}} = P_{\text{base}} + (\Delta t_{\text{espera}} \times \alpha)$$

donde $P_{\text{base}}$ corresponde a la ponderación base del nivel (100 para Nivel 1, 20 para Nivel 5), $\Delta t_{\text{espera}}$ es el tiempo transcurrido en la cola virtual y $\alpha$ es el factor de envejecimiento. Con el paso del tiempo, un pedido económico incrementa su prioridad efectiva hasta superar a pedidos prioritarios de reciente llegada.

3. Menor Cantidad de Unidades Primero (Shortest Job First - SJF): Utiliza la cantidad total de artículos del pedido como la estimación directa de la ráfaga de trabajo a realizar:

$$t_{\text{trabajo}} = \text{unidades} \times 100 \text{ ms}$$

Al despachar primero los pedidos más pequeños, se liberan estaciones y personal con mayor velocidad, minimizando el tiempo promedio global de espera del centro.

4. Primero en Llegar, Primero en Atender (FIFO): Ordenamiento cronológico puro basado en el timestamp de creación del pedido, garantizando orden de llegada sin distinciones de urgencia o tamaño.

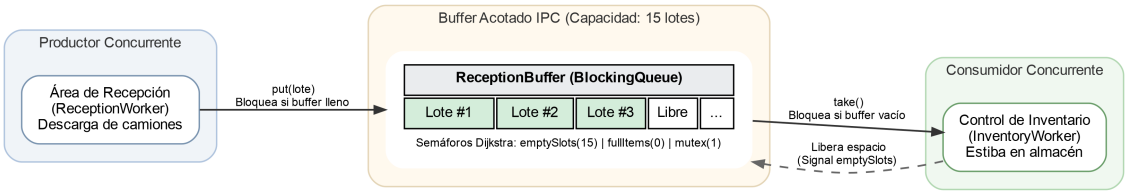
### 5.3. Niveles de Servicio y SLAs Contractuales

| Nivel | Tipo de Envío | Tiempo Objetivo (SLA) | Prioridad Base | Perfil Típico |
|---|---|---|---|---|
| 1 | Exprés | Casi instantáneo (1 min) | 100 | Urgencias médicas e insumos críticos |
| 2 | Prioritario | Máximo 10 minutos | 80 | Clientes premium y corporativos |
| 3 | Estándar | Máximo 30 minutos | 60 | Comercio electrónico regular |
| 4 | Programado | Máximo 60 minutos | 40 | Despachos con entrega agendada |
| 5 | Económico | Máximo 120 minutos | 20 | Carga consolidada de bajo costo |

## 6. Comunicación entre Procesos (IPC) con Buffer Acotado

El centro de distribución requiere la coordinación permanente entre el área de Recepción de Mercadería y el área de Control de Inventario. El enunciado exige explícitamente que estas áreas funcionen de forma independiente y no modifiquen de manera directa las estructuras de datos de la otra.

### 6.1. Modelo Productor-Consumidor sobre Buffer Acotado




Esquema de comunicación asíncrona mediante buffer acotado.

### 6.2. Mecanismos de Sincronización y Contrapresión

La clase ReceptionBuffer encapsula una cola bloqueante con capacidad acotada a quince elementos. Las operaciones están sincronizadas para garantizar dos invariantes fundamentales:

1. Prevención de Desbordamiento (Overflow): Si los camiones de recepción entregan mercadería a un ritmo superior a la velocidad de clasificación de inventario, el buffer alcanza su capacidad máxima y suspende temporalmente al hilo productor sin pérdida de datos ni corrupción de memoria.
2. Prevención de Espera Activa (Busy Waiting): Si el buffer se encuentra vacío, el hilo consumidor de inventario se suspende eficientemente sobre la condición correspondiente hasta que un nuevo lote es insertado por recepción.

Una vez que el lote es extraído, el trabajador de inventario actualiza el stock físico en el catálogo y dispara el evento reevaluateWaitingOrders del planificador, permitiendo que los pedidos que estaban bloqueados en estado de espera de inventario pasen inmediatamente a la cola de espera de recursos.

## 7. Almacenamiento Físico y Asignación No Contigua (Paginación)

Una de las analogías más potentes con los conceptos de sistemas operativos es la administración física del almacén. En un sistema de archivos o en memoria virtual con paginación, un proceso de gran tamaño no requiere ocupar un bloque contiguo de memoria física, sino que se dispersa a lo largo de marcos de página libres.

### 7.1. Estructura Físico-Espacial del Almacén

El almacén está modelado por la clase WarehouseGrid mediante una matriz discretizada en coordenadas tridimensionales:

- 5 Pasillos numerados del 1 al 5.
- 4 Estantes por pasillo (Estante 1 al 4).
- 3 Niveles verticales por estante (Nivel 1 al 3).
- 5 Casillas por nivel (Casilla 1 al 5).

El espacio total disponible equivale a:

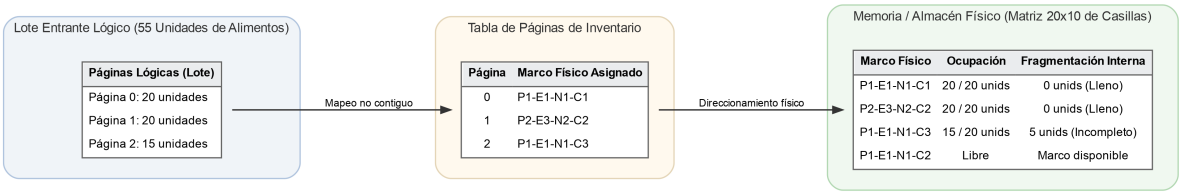
$$\text{Total Casillas} = 5 \times 4 \times 3 \times 5 = 300 \text{ casillas físicas (Page Frames)}$$

Con una capacidad fija de 20 unidades por casilla, la capacidad volumétrica total asciende a 6,000 unidades.

### 7.2. Algoritmo de Fragmentación y Tabla de Asignación

Cuando ingresa un lote de $N$ unidades de producto:

1. El algoritmo recorre las casillas buscando espacios disponibles que ya contengan el mismo producto o casillas totalmente vacías.
2. No exige que las casillas sean contiguas en el mismo pasillo o nivel.
3. Asigna porciones del lote en distintas ubicaciones dispersas (por ejemplo, 20 unidades en Pasillo 1-Estante 1, 20 en Pasillo 2-Estante 3 y 10 en Pasillo 4-Estante 2).
4. Retorna una lista de objetos StorageAllocationEntry, la cual funciona exactamente como la Tabla de Páginas del lote.




Representación esquemática de la asignación no contigua de un lote de 55 unidades.

Al momento del despacho de un pedido, las unidades son extraídas de sus casillas correspondientes. Cuando una casilla llega a cero unidades ocupadas, se libera automáticamente y regresa al pool de espacio disponible.

# 8. Conflicto de Recursos: Detección y Resolución de Deadlock

Bajo condiciones de contención severa, la solicitud concurrente de recursos compartidos en orden inverso puede inducir una condición de interbloqueo o punto muerto (Deadlock).

## 8.1. Las Cuatro Condiciones de Coffman

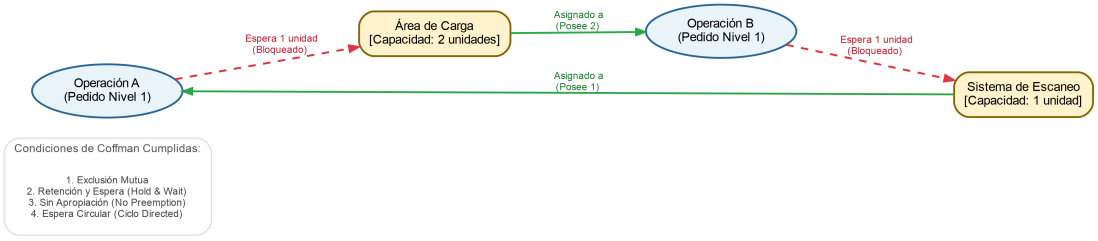
Para que ocurra un interbloqueo en REDXela deben manifestarse de forma simultánea las cuatro condiciones teóricas enunciadas por E. G. Coffman:

1. Exclusión Mutua: Los recursos críticos, como el Sistema de Escaneo de Códigos y las Áreas de Carga, no pueden ser compartidos simultáneamente por dos operaciones incompatibles.
2. Retención y Espera (Hold and Wait): Una operación adquiere un primer recurso y lo retiene mientras solicita un segundo recurso que actualmente está en posesión de otra operación.
3. Sin Apropiación (No Preemption): El sistema no puede arrebatarle arbitrariamente un recurso a un hilo; sólo la operación que lo posee puede liberarlo voluntariamente.
4. Espera Circular (Circular Wait): Existe una cadena cerrada de procesos $\{P_A, P_B\}$ tal que $P_A$ espera un recurso retenido por $P_B$, y $P_B$ espera un recurso retenido por $P_A$.

## 8.2. Grafo de Asignación de Recursos y Detección de Ciclos

La clase WaitForGraph modela las dependencias activas del sistema. Las aristas de asignación indican posesión (Recurso $\rightarrow$ Operación), mientras que las aristas de solicitud indican contención bloqueada (Operación $\rightarrow$ Recurso).

El detector evalúa periódicamente la existencia de ciclos utilizando una búsqueda en profundidad (DFS). Al detectar un ciclo dirigido cerrado, se instancia un objeto DeadlockIncident y se notifica al Event Dispatch Thread para suspender las operaciones en conflicto y abrir el diálogo de resolución manual.




Ciclo de espera circular provocado deliberadamente entre dos operaciones y dos recursos.

### 8.3. Protocolo de Resolución Manual

Cuando el usuario interviene a través del diálogo interactivo:

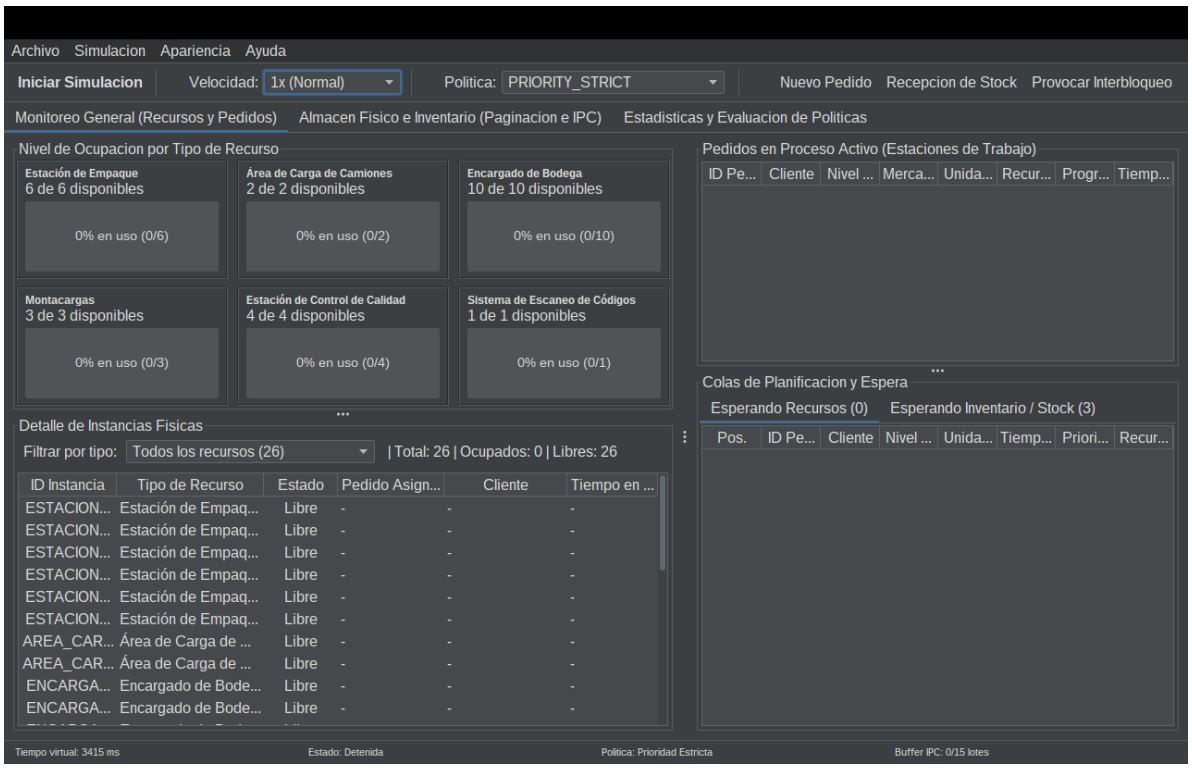
1. El usuario selecciona cuál de las dos operaciones en disputa debe continuar (por ejemplo, Operación A).
2. La operación elegida recibe la prioridad para adquirir el recurso disputado.
3. La operación no seleccionada (víctima):
    - Es interrumpida de manera segura en su llamada bloqueante.
    - Ejecuta su rutina de liberación, devolviendo todos los permisos retenidos al semáforo correspondiente.
    - Su estado transiciona a EN_ESPERA_RECURSOS.
    - Es reingresada en la cola de planificación para ser atendida posteriormente cuando los recursos queden libres.
4. Se limpian las aristas correspondientes en el grafo de espera y la simulación reanuda su marcha normal.

## 9. Interfaz Gráfica de Usuario y Soporte LookAndFeel

La interfaz de usuario fue diseñada bajo la estricta directriz de emplear componentes estándar de Java Swing nativo sin colores ni bordes rígidos cableados en código, garantizando compatibilidad inmediata con la biblioteca FlatLaf en modos claro y oscuro.

### 9.1. Pestaña 1: Monitoreo General (Recursos y Pedidos)

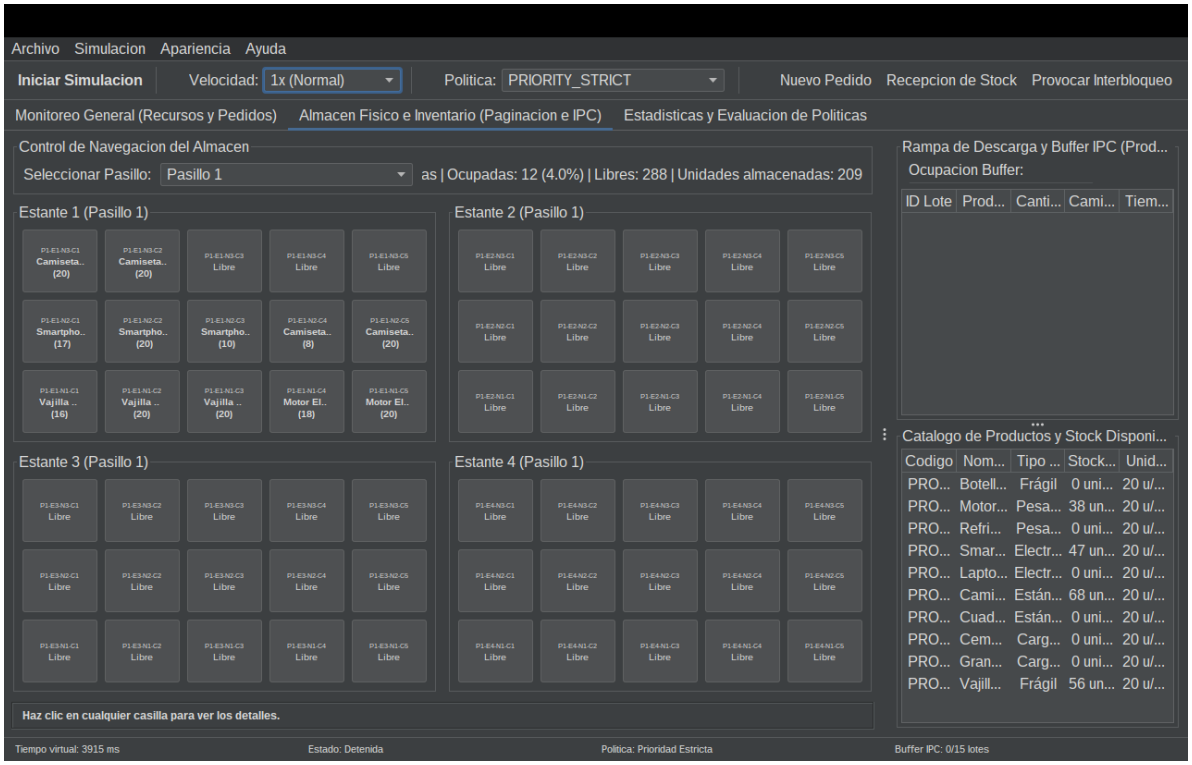
Esta vista presenta en su panel izquierdo seis tarjetas ejecutivas con el porcentaje de ocupación de cada recurso finito y una tabla maestra inferior filtrable para inspeccionar individualmente las 26 instancias físicas. En el panel derecho se ubica la tabla de pedidos que están siendo atendidos en tiempo real por los trabajadores, complementada con pestañas para las colas de espera.




Monitoreo general de ocupación de recursos y seguimiento de pedidos en proceso.

## 9.2. Pestaña 2: Almacén Físico e Inventario

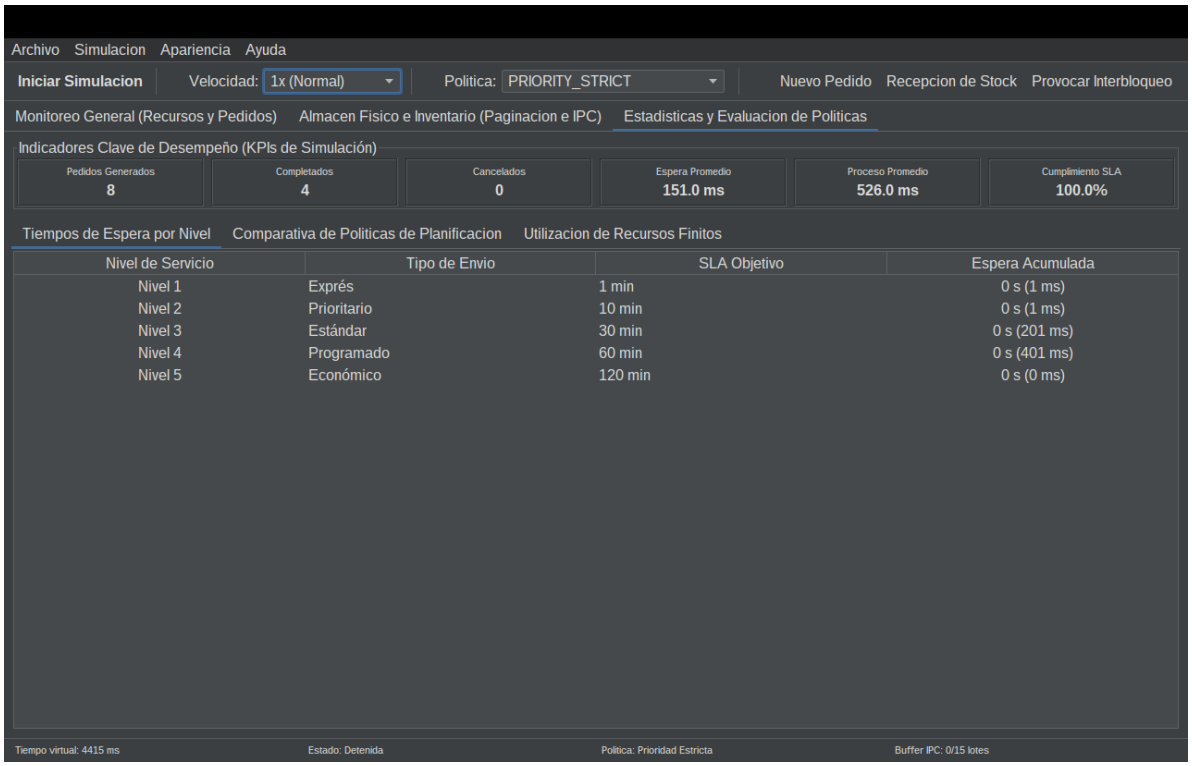
El visualizador organiza el almacén seleccionando el pasillo activo y mostrando sus cuatro estantes en una cuadrícula armónica. Cada botón representa una casilla física con indicación de ocupación y código espacial. El panel derecho muestra el estado del buffer acotado de recepción y el catálogo de inventario disponible.




Visualizador del almacén físico con asignación no contigua y buffer de recepción.

## 9.3. Pestaña 3: Estadísticas y Evaluación de Políticas

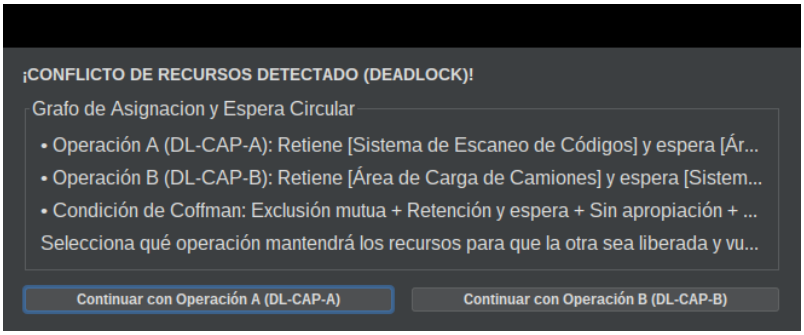
Despliega un tablero de indicadores clave de desempeño (KPIs) en tarjetas métricas superiores, vinculadas con el reloj virtual de la simulación. En la parte inferior, tablas comparativas desglosan el tiempo de espera acumulado por nivel de servicio, la cantidad de pedidos completados por cada política de atención y la utilización promedio de los recursos compartidos.



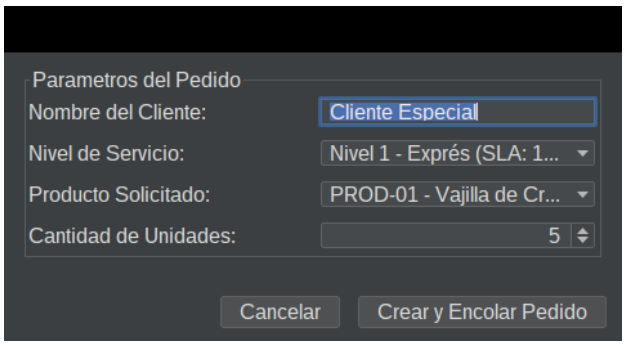

Tablero de control estadístico y evaluación de cumplimiento de SLAs.

## 9.4. Diálogos Modales del Sistema

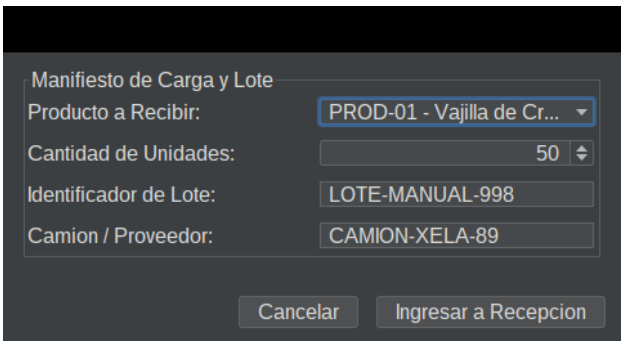
El sistema cuenta con tres ventanas de diálogo modales diseñadas para interactuar con la simulación en tiempo real:




Diálogo de detección y resolución interactiva de interbloqueo.




Ingreso manual de pedido.




Recepción manual de mercadería.

# 10. Estrategia de Pruebas y Verificación Automatizada

La estabilidad de los algoritmos de concurrencia se verificó mediante una batería de pruebas automatizadas con JUnit 5, ejecutables mediante el comando estándar de Maven:

```
mvn clean test
```

## 10.1. Matriz de Pruebas Unitarias y de Integración

| Clase de Prueba | Concepto Evaluado | Criterio de Aceptación |
|---|---|---|
| ConcurrencyTest | Exclusión mutua de recursos | 30 hilos compiten por 6 estaciones de empaque. Nunca se superan 6 accesos simultáneos y los permisos retornan a 6 al concluir. |
| WarehouseAllocationTest | Almacenamiento no contiguo | Un lote de 100 unidades se fragmenta en 5 casillas dispersas. La desasignación devuelve las casillas a estado libre sin fugas de capacidad. |
| IPCBufferTest | Buffer acotado (IPC) | Productor veloz y consumidor lento sobre buffer de 10 elementos. Comprueba ausencia de desbordamiento y cero pérdida de datos. |
| DeadlockDetectionTest | Grafo RAG y ciclos | Dos hilos disputan recursos en orden inverso. WaitForGraph detecta el ciclo inmediatamente y la resolución libera la víctima. |
| SchedulerPolicyTest | Políticas de planificación | Valida matemáticamente el ordenamiento para Prioridad Estricta, Aging dinámico, Shortest Job First y FIFO. |
| MainFrameUITest | Integridad gráfica | Instanciación limpia de todos los paneles y verificación del ciclo de refresco en el EDT sin excepciones. |
| REDXelaSystemIntegrationTest | Integración de extremo a extremo | Ciclo de vida completo: llegada de stock, estiba física, encolado, cambio de políticas, despacho y métricas. |

## 10.2. Resultado de la Ejecución de Pruebas

Todas las pruebas ejecutadas en el entorno automatizado finalizaron con un 100 % de éxito:

```
Results: Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

## 11. Conclusiones Técnicas y Recomendaciones

1. La aplicación rigurosa de semáforos con equidad y bloques de liberación estructurados garantiza que el centro de distribución maneje altos volúmenes de pedidos concurrentes sin riesgo de condiciones de carrera ni fugas de permisos.

2. La separación entre áreas operativas mediante canales IPC con buffer acotado demuestra cómo la contrapresión previene la saturación de memoria cuando los ritmos de producción y consumo difieren sustancialmente.

3. La simulación de almacenamiento no contiguo ilustra con claridad didáctica el concepto de marcos de página y tablas de asignación de memoria virtual aplicados a un escenario logístico tangible.

4. La detección de ciclos en el Grafo de Asignación de Recursos mediante búsqueda en profundidad permite no solo evidenciar el cumplimiento de las cuatro condiciones de Coffman, sino ofrecer un mecanismo pedagógico e interactivo para su resolución manual sin comprometer la integridad de la simulación.

5. La construcción de la interfaz sobre componentes Java Swing 100 % nativos asegura una integración estética limpia y profesional con FlatLaf, manteniendo el código libre de estilos propietarios que dificulten su mantenimiento o portabilidad.